Big Data Analytics Programming
Assignment 3: *Locality Sensitive Hashing*

Pieter Robberechts
pieter.robberechts@kuleuven.be

Projects are independent: no working together! You must come up with how to solve the problem independently. Do not discuss specifics of how you structure your solution, etc. You cannot share solution ideas, pseudocode, code, reports, etc.

Beyond what is provided, you may not use any other code that is available online. The use of language models such as ChatGPT or any other AI-based systems for completing this programming assignment is strictly prohibited. If you are unsure about the policy, ask the professor in charge or the TAs.

1 Problem statement and motivation

Finding sets of similar objects in large collections is a fundamental problem in computer science. One of the most prominent contemporary use cases is the deduplication of massive text corpora used to train Large Language Models (LLMs) [LIN+22, GBB+20]. When scraping the internet to build training datasets, researchers ingest millions of duplicate and near-duplicate web pages (e.g., mirrored sites, boilerplate text, and repeated open-source licenses).

If an LLM is trained on this redundant data, two major problems occur. First, processing the same documents repeatedly wastes millions of dollars in GPU compute. Second, it causes "memorization": the model stops learning general language patterns and instead memorizes the exact duplicated text, which can lead to severe copyright or privacy violations when the model generates responses.

The problem of finding these near-duplicates can be specified as follows: given a similarity function sim that maps pairs of objects to a numeric score, find all the pairs of objects (x, y) such that $sim(x, y) > t$, where t is a user-defined similarity threshold.

The problem can be solved with the following (naive) algorithm.

Input:
A collection $C = \{x_1, \ldots, x_n\}$ of objects
A similarity function $sim : C \times C \to \mathbb{R}$
A minimum similarity threshold t
Output:
The set S of all pairs (x_i, x_j) such that $sim(x_i, x_j) > t$

$S \leftarrow \emptyset$ ▷ Initialize an empty set
for all $i \in \{1, \ldots, n\}$ **do**

```
    for all j ∈ {i + 1, ..., n} do
        if sim(x_i, x_j) > t then
            S ← S ∪ {(x_i, x_j)}
        end if
    end for
end for
return S
```

The complexity of this algorithm is quadratic in the number of objects in the collection. Such a complexity is acceptable for simple problems, but quickly becomes unpractical as the number of objects grows. For billion-document web corpora, computing the similarities using this algorithm is mathematically impossible.

Locality sensitive hashing (LSH) is an approximate method that can be used to find pairs of objects with high similarity more efficiently. It hashes the most similar objects into the same buckets, and most dissimilar objects into different buckets. Approximate similarity can then be computed only for those pairs of objects that are hashed into the same bucket. If the hashing function is properly designed, this yields a drastic reduction in runtime.

More details and a complete description of the LSH procedure can be found in the book *Mining of Massive Datasets* by Jure Leskovec, Jeffrey David Ullman and Anand Rajaraman. The book is freely available online and the corresponding chapter about LSH can be found at: `http://infolab.stanford.edu/~ullman/mmds/ch3.pdf`. Read this chapter carefully before you continue reading the following sections.

2 Applying LSH to LLM Training Data

For this assignment, you will implement LSH and apply it to a dataset containing raw web documents to simulate an LLM data deduplication pipeline.

2.1 Dataset

We will consider one million documents, obtained from the "noclean" variant of the C4 dataset (Colossal Clean Crawled Corpus), which is widely used to train language models. A preprocessed subset of this dataset is available in `/cw/bdap/assignment3/`.

The dataset is formatted as JSON Lines (`.jsonl`), meaning every line in the file is a valid, independent JSON object representing a single web document. Each object contains several fields, including `"url"`, `"timestamp"`, and `"text"`. For this assignment, your LSH algorithm should process the `"text"` field to calculate similarity. To identify the documents in your final output, you may use the document's `"url"`.

2.2 Implementation [16 pts]

Your task is to implement the LSH algorithm, and run it on the C4 data to find similar pairs of documents. You are given code that is able to read the documents, and some helper classes. More specifically, you are given the following files (in `assignment3.tgz` on Toledo):

- `Reader.java`: base class for a document reader
- `DocumentReader.java`: reads and parses the JSONL web documents

- `Shingler.java`: maps documents to a shingle representation
- `MurmurHash.java`: hash function used in the Shingler, might also be used in the LSH implementation
- `Primes.java`: used to obtain the least prime number greater than a certain value, needed for constructing a hash table
- `SimilarPair.java`: convenience class to represent similar pairs
- `Runner.java`: allows to run a similarity search from the command line
- `BruteForceSearch.java`: implementation of the naive algorithm for finding similar documents

These classes are complete and should not be changed, except for `Shingler.java` and `Runner.java`, which may need to be adapted to use your chosen internal data structures (see below).

Furthermore, you are given class stubs for implementing the LSH algorithm:

- `SimilaritySearcher.java`: base class for a similarity search algorithm
- `Minhash.java`: methods for computing the minhash signatures
- `LSH.java`: implementation of the LSH algorithm for finding similar documents

These classes are incomplete and use Java generics to allow you to decide on the most efficient internal data structures:

- `<T>` represents the data structure for a single document's **shingle set**.
- `<S>` represents the data structure for the **signature matrix**.

You must specify your chosen concrete types in `Runner.java` and adapt `Shingler.java` to return shingles in the appropriate data structure.

In order to make a functional algorithm, you must at least complete the methods that are tagged with "THIS METHOD IS REQUIRED". The headers of all the methods, including the constructor, should not be changed. The rest of the code may be changed as long as all the required methods work as expected. You can add methods and classes if needed.

These class stubs allow us to test the correctness of your code automatically but can be quite restrictive. If you believe that the parameter or return types do not allow the most efficient implementation, you can add additional methods (e.g., `Minhash.constructHashTableFast`) or classes (e.g., `LSHFast`) and call these instead when running your program. However, note that you still have to implement the original methods since these are required to pass our automated tests.

Your code is graded on (1) correctness, (2) efficiency, and (3) readability and documentation. Note that when evaluating correctness, we can test your code on edge cases that might not occur while running on the given dataset. We use a predefined (non-disclosed) set of parameters to evaluate the time efficiency.

You can use multithreading as an optimization. However, this is not something we generally expect you to implement and we will evaluate the performance on a single core anyway. The focus of this assignment is the LSH algorithm and handling memory constraints gracefully.

2.3 Compiling and running code

Once your implementation is complete we should be able to run your code with the following command:

```
java Runner -method bf
        -dataFile path/to/dataset
        -outputFile path/to/output.tsv
        -threshold 0.5
        -maxDocs 1000
        -shingleLength 5
        -numShingles 2500
```

Where:

- `-method bf` *or* `lsh`: method used. Either `bf` for *brute force* or `lsh` for *locality sensitive hashing*.
- `-dataFile` *path*: is the path to the data file with documents.
- `-outputFile` *path*: discovered similar pairs are written to this file.
- `-treshold` *double*: similarity threshold.
- `-maxDocs` *int*: is the maximum number of documents to consider. If set to n, `Reader.java` will only load documents $0, \ldots, n-1$.
- `-shingleLength` *int*: length of the shingles.
- `-numShingles` *int*: maximum number of unique shingles.
- `-seed` *int*: seed for the random generator. Runs with the same seed must produce identical results.

For the LSH method, the following parameters are also available:

- `-numHashes` *int*: the number of hash functions for the signature matrix.
- `-numBands` *int*: the number of bands.
- `-numBuckets` *int*: the number of buckets.

Running this command should produce a TSV output file containing the pairs with a similarity above the given threshold. Each line has the following format: `DocIdentifier1 \t DocIdentifier2 \t similarity`. (Use the `url` field for the document identifiers).

For convenience, a Makefile is provided that can be used for compiling, testing and running the code. Using this Makefile is optional. You are not required to submit one and you can adjust the provided Makefile for running experiments as you wish. Note that the parameters above and in the Makefile are randomly picked. You'll have to tune them.

2.4 Experiments

Find a good parameter configuration for obtaining the pairs of documents in the full dataset that have a **Jaccard similarity > 0.8** when using **9-shingles** at the word level with a time budget of 30 minutes (i.e., the total runtime should not exceed 30 minutes). There is no single correct solution for determining the parameters. You should pick a configuration that provides a reasonable trade-off between speed and accuracy. E.g., if it takes 90% longer to gain 0.01% in accuracy, it might not be worth it. The most important part is that you can motivate your experimental strategy and final choice in the report. We expect a discussion

of extensive experimentation. Parameter tuning is a crucial aspect of optimizing performance in big data analytics. You should be able to show that you can do this systematically and interpret the results correctly.

You should provide the parameter configuration through a command given in a `command.sh` file contained in your solution. This file should contain a single line with a command of the following form:

```
java Runner -method lsh -threshold 0.8 -shingleLength 9 ...
```

Please do not include code to compile your code, no variable substitutions, no classpath, etc.

Besides the arguments that are specified in the command above, you should also expose all other parameters that are important to the runtime or solution quality of your program (for example, we would certainly expect to see parameters specifying the number of bands and the number of rows per band).

The output file should contain all the pairs of items that are identified as having a similarity > 0.8 by your LSH program (this file can contain false positives). The similarity listed in the output file does not have to be exact (i.e., it can be computed based on the signature matrix).

Note that the memory limit for remote (ssh) users of the departmental machines is 2 GB. This is a critical limitation for big data applications! Because 1 million web documents cannot fit into 2 GB of RAM alongside their signatures, you must take this into account while developing your solution. Streaming the data and managing memory efficiently is just as important as choosing the right parameters.

3 Report [4 pts]

Your report should contain (1) a discussion of any important (w.r.t. runtime and/or memory efficiency) choices that you had to make in developing your solution, (2) a thorough description of the experiments performed as discussed in 2.4, and (3) a brief motivation for the specific set of parameters chosen to run on the full dataset, as well as the runtime of your final solution. The report can be a maximum of 2 pages including any tables or figures. Exceeding the page limit will be penalized. You can use the following comment to set margins in Latex:

```
\usepackage{anysize}
\marginsize{15mm}{15mm}{15mm}{15mm}
```

Please pay attention to standard best practices such as labeling any axes, using an appropriate number of significant digits, including units of measure (e.g., if you report run time, you should say if it is in seconds, minutes, hours). It should also not include things such as screenshots of your terminal.

4 (Very) Important remarks

Deadline The assignment should be handed in on Toledo before **May 29$^{\text{th}}$ at 17h00**. We will not accept late submissions!

Deliverables You must upload an archive (`.zip` or `.tar.gz`) containing the results on the full dataset (as `.tsv`), the command used to generate the results (as `.sh`) and a folder with all the source files using the file hierarchy below:

```
assignment3/
├── report.pdf
├── output.tsv
├── command.sh
└── src/
    ├── SimilaritySearcher.java
    ├── LSH.java
    └── ...(all source files needed to compile and run your solution)
```

The `.java` files should contain complete source code for the brute force and the LSH algorithms. Files included in the template that were not modified do not have to be included in your submission. The `output.tsv` file should contain the output file for running your LSH implementation on the full dataset.

Automated grading Automated tests are used for assessing the correctness. Therefore you must follow these instructions:

- Use the exact filenames as specified in the assignment.
- Do not change the provided interface/skeletons, this includes the constructor.
- Your code must run on the departmental machines, this can be done over ssh (see documents of exercise session 0).
- You should not implement any additional data preprocessing (as we will check your solution for correctness, which requires using the exact same input data).
- Never use absolute paths.
- Use the seed provided via the command line argument when generating random numbers.

If you fail to follow these instructions, you will lose points.

Running experiments This is a class about big data, so running experiments could take hours or even multiple days. You can do this easily on the departmental machines: it takes several minutes to start the experiments and then you check back later to see the results. Failure to run and report results on the full data set will result in a substantial point reduction.

Sharing code or data The provided source code cannot be redistributed in any form, including but not limited to uploading it to public repositories, posting it on forums or sharing it with other individuals or organizations.

Questions A FAQ will be kept up to date on Toledo. If anything is still unclear, you can contact `pieter.robberechts@kuleuven.be`.

Good luck!

References

[GBB+20] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020.

[LIN+22] Katherine Lee, Daphne Ippolito, Andrew Nystrom, Chiyuan Zhang, Douglas Eck, Chris Callison-Burch, and Nicholas Carlini. Deduplicating training data makes language models better. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 8424–8445, 2022.